

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Ensemble Learning – Research Project

Project 1: report

Non-linear and Interaction Features for Deep Forest

Authors:

Wojciech Jasiński

Sandra Adamiec

Warsaw, 12 November 2026

Abstract

Deep Forest builds a model from layers of random forests: each layer sees the input features and the predictions of the layer before it. We tested whether a layer does better when it also gets a few extra columns: products of two features, products of a feature and a prediction, measures of how unsure the forests are, or a sigmoid of the predictions. We fixed five hypotheses and the tests that decide them before any run, and compared 11 methods on 32 public datasets, 16 for classification and 16 for regression.

Main findings

- None of the five hypotheses is supported. No kind of added column beat the plain cascade, and a typical gain above about 0.7% is unlikely for any of them.
- The idea works on data built for it. When the target contains an exact product of two features, the products find it and cut the error by about 40%.
- Follow-up runs (mostly one seed, chosen after the results) suggest why it fails on real data: extra columns make the forests slightly worse, most of all when they are pure noise, and the products do not bring enough to make up for it.
- Tuned LightGBM ranks best of the 11 methods. The plain cascade beats a single random forest with as many trees as one layer.

1 Introduction

Deep Forest [3] is a tree-based model built in layers. Each layer holds a few random forests. It receives the original features plus the predictions of the layer before it (class probabilities, or predicted values in regression), and the final answer comes from the best layer (Figure 1).

This design has two weak spots. The predictions are the only new information a layer gets. And a tree splits on one column at a time, so it can miss two features that matter only together, for example when the class depends only on whether two features have the same sign [5]. The course topic asked for non-linear features or features that combine two or more features. We tried both, by giving each layer a few extra columns.

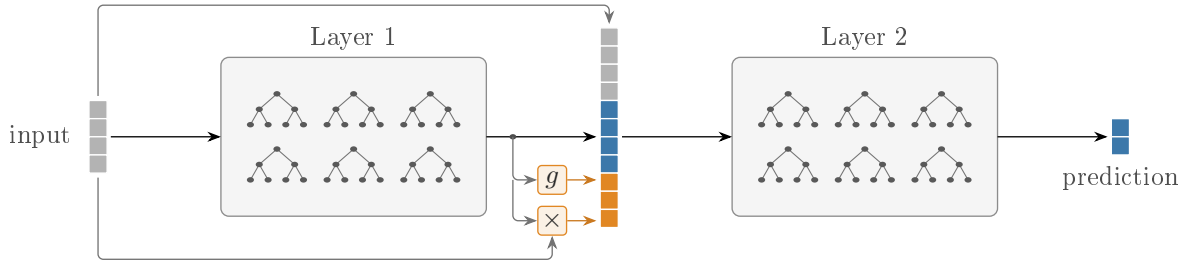


Figure 1: Deep Forest with our change, first two layers. A layer gets the input features (■) and the previous layer's predictions (■). We add new columns (■): functions g of the predictions, and products $\times$ of two features or of a feature and a prediction. Feature–feature products also enter the first layer (not shown).

Related work. Others have changed what Deep Forest layers pass on. hiDF [6] replaces the predictions with high-order interactions, MDGP-forest [7] builds new features between layers by genetic programming, and MVU-DF [8] passes an uncertainty column together with several other changes. Our way of choosing products comes from Reluctant Interaction Modeling [9]: fit the main effects first, then add only the interactions that

correlate with what the model still gets wrong. As far as we know, nobody has tested such columns on their own in Deep Forest, against random and target-chosen products and against a single forest.

This report covers both the plan and the results, and leaves out most details. The plan [1] and the long results report [2] have them.

2 What we tested

Every variant is the *base cascade*, our Deep Forest without added columns, plus one kind of extra column. Nothing else changes. We wrote five hypotheses into the plan before running anything:

- **H1 Feature products help.** Products of two features, chosen by how well they correlate with the previous layer's error, lower the error.
- **H2 The choice matters.** These products do better both than the same number of products picked at random (H2a) and than products picked by correlation with the target (H2b).
- **H3 Feature–prediction products help.** Products of a feature and a prediction, chosen by mutual information with the target, lower the error.
- **H4 Uncertainty helps.** Columns that say how unsure the forests are and how much they disagree lower the error.
- **H5 Monotone activations help.** Order-preserving functions of the predictions, such as the sigmoid, ReLU or GELU, lower the error.

The added columns. Products are computed from standardized features. Feature–feature (XX) products enter every layer; feature–prediction (XP) products and the other columns start at layer 2, because the first layer has no predictions yet. A product variant adds K products: no more than the number of features, and at most a quarter of all feature pairs. Table 1 lists what each method adds. *Residual-XX*, our main proposal, picks the pairs whose product correlates most with the *layer error*: the true label minus the layer's average prediction, that is, the residual. At layer 1, which has no error to use yet, it picks by correlation with the target, and from then on it picks again at every layer. *XP-MI* ranks feature–prediction products by mutual information, a measure of how much a column tells about the target. *Non-linear* adds three columns: how unsure each of the two forest types is (the entropy of its class probabilities, or in regression the spread of its trees' predictions) and how much the two types disagree. *Monotone* adds a sigmoid of every prediction column.

3 How we tested

Cascade. Our cascade follows the official Deep Forest package, DF21 [4]. Each layer has two random forests and two extremely randomized forests of 100 trees each. Layers are added, up to 20, until two in a row bring no improvement, and the model predicts with the best layer. Two things differ from DF21, both planned. The predictions a layer passes on for its training rows come from 3-fold cross-validation inside the training set (we call these *inner folds*), not from out-of-bag estimates. And the best layer is chosen by balanced accuracy, not accuracy. DF21 cannot add columns, so we rewrote the cascade in

Table 1: The 11 compared methods. XX: products of two features; XP: products of a feature and a prediction.

Method	Added columns	Chosen by	Role
Base cascade	none	–	reference
Random-XX	K XX products	random, new at every layer	control (H2a)
Target-XX	K XX products	correlation with the target, same in every layer	control (H2b)
Residual-XX	K XX products	correlation with the layer error	tested (H1, H2)
XP-MI	K XP products	mutual information with the target	tested (H3)
Non-linear	3 uncertainty columns	fixed	tested (H4)
Monotone	sigmoid of each prediction	fixed	tested (H5)
Combined	non-linear and residual-XX	as in their own rows	both together
RF	none	–	one forest, 400 trees
RF + target-XX	K XX products	correlation with the target	products, no cascade
LightGBM [10]	none	–	tuned booster

scikit-learn. On two regression sets it matched the package; on two classification sets its balanced accuracy was 0.5 and 2.9 points lower. Our conclusions are about this cascade. All settings are listed in Table A1.

Data. We used 32 public datasets, 16 for classification and 16 for regression, with about 100 to 49,000 rows (Table A2). The largest ones were subsampled to 20,000 rows. We removed identifiers, timestamps and columns that give the target away. Missing values were filled in, categorical features one-hot encoded and numeric ones standardized, always fitted on the training rows only.

Runs. Each dataset was split into 5 folds three times, with different seeds: 15 train/test runs per method. Rows that belong together, such as the same patient, always stay in one fold, and all methods use the same splits. The error is 1 − balanced accuracy in classification and RMSE in regression. The cascade's settings are not tuned and are the same for every variant; only LightGBM is tuned (10 random settings on 3 inner folds). The main experiment had 5,280 runs. Eight sensitivity checks, each changing one setting, added 3,840 runs with one seed. None failed.

Deciding a hypothesis. For each dataset we compute the *relative error reduction* of a variant against the method it is compared with,

$$r = \frac{\bar{e}_{\text{ref}} - \bar{e}}{\max(\bar{e}_{\text{ref}}, \bar{e})},$$

where $\bar{e}$ is the mean error over the 15 runs. So $r = 5\%$ means the variant's error is 5% lower, and $r = -5\%$ that the reference's error is 5% lower. A Wilcoxon signed-rank test over the 32 datasets then asks whether the variant wins more often, and by more, than chance would explain. Because there are six such tests, each is made stricter with Holm's correction [11]. A hypothesis is supported only if its corrected test passes ($p < 0.05$) and the median reduction is positive; H2 needs both of its comparisons.

4 Results

4.1 At a glance

None of the five hypotheses is supported (Table 2). Figure 2 shows why. For every comparison the datasets fall on both sides of zero, and every median is within 1% of zero. For the four kinds of column compared with the base cascade (H1, H3–H5), a typical gain above about 0.7% is unlikely on data like ours.

Table 2: The six comparisons that decide the hypotheses (32 datasets). Better/worse/tie: datasets where the first method's mean error is lower, higher or equal. Median: median relative error reduction (%), positive favours the first method. p: Wilcoxon test with Holm's correction. A hypothesis needs $p < 0.05$ and a positive median; none meets this.

	Comparison	Better/worse/tie	Median	p
H1	Residual-XX vs base	14/18/0	−0.66	0.47
H2a	Residual-XX vs random-XX	16/16/0	0.02	1.00
H2b	Residual-XX vs target-XX	20/9/3	0.29	0.12
H3	XP-MI vs base	9/17/6	−0.06	0.81
H4	Non-linear vs base	14/12/6	0.00	1.00
H5	Monotone vs base	8/18/6	−0.20	0.47

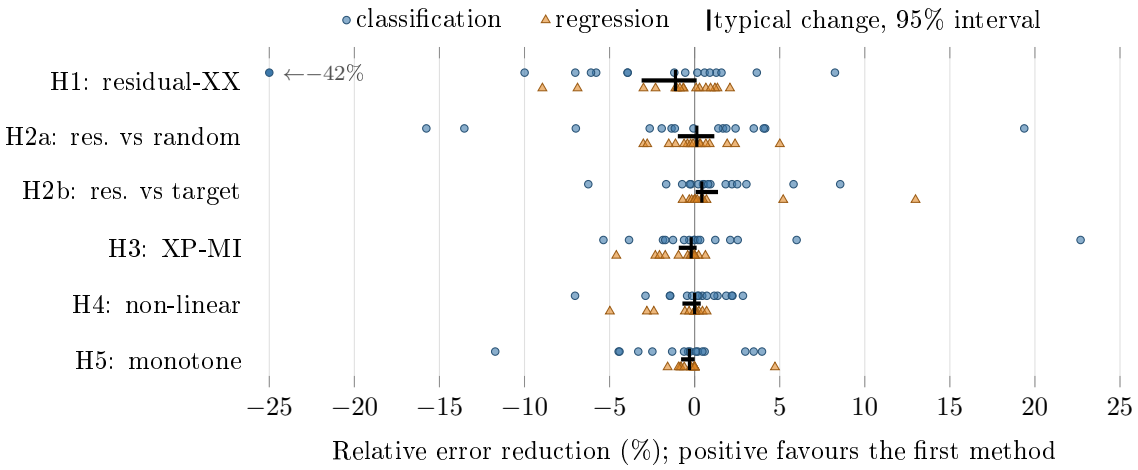


Figure 2: Relative error reduction on each dataset for the six comparisons of Table 2; one dot per dataset. Black: a robust estimate of the typical change (Hodges–Lehmann [12]) with its 95% interval. The intervals are not corrected for the six tests, and the estimate can differ from the median in Table 2. One dot lies off the axis: residual-XX on Tic-Tac-Toe, −42% (error 2.3% instead of 1.4%).

4.2 H1: Do feature products help?

No. Residual-XX made the cascade slightly worse: its error was higher than the base cascade's on 18 of the 32 datasets. Dataset by dataset (a paired test over the 15 runs), it was better on one and worse on ten. The follow-up runs in Section 6 suggest why: extra columns have a cost, and on real data the products do not bring enough to cover it.

4.3 H2: Does the way of choosing products matter?

Not much. Choosing products by the layer error did no better than choosing them at random. It beat choosing by the target more often than not, but this does not survive the correction for six tests and shows up clearly in only one of the three seeds. Its largest win, on Bike Sharing, is target-XX failing: it kept the same products at every layer and lost 15% to the base cascade. No way of choosing beat adding no products at all (see the ranking in Figure 3).

4.4 H3: Do feature–prediction products help?

No. The median change is close to zero. On six datasets nothing changed at all, mostly because the cascade predicted from layer 1 in every run there, before these columns come in. In regression XP-MI was worse more often than better (9 datasets to 2; not a planned test). It is also the most expensive of the tested variants: a median run takes 46% longer than the base cascade.

4.5 H4: Do uncertainty columns help?

No clear effect either way. The median change is zero: some datasets got better and about as many got worse. In the ranking (Figure 3) the variant sits right next to the base cascade. The columns are cheap, a few percent more run time, and adding them next to the products showed no clear benefit either.

4.6 H5: Do monotone activations help?

No, and they seem to make class probabilities worse. The median change is small and balanced accuracy hardly moves, but ROC AUC and log-loss get worse on 14 and 15 of the 16 classification datasets. This is not one of the planned tests, so read it as a hint. A likely reason: the forests bin every column into quantiles, and a monotone function keeps the order of values. After binning, the sigmoid of a prediction therefore splits the training rows exactly like the prediction itself, so the new columns are mostly copies. We suspect the copies give the predictions extra weight and make the forests overconfident. Without binning, the sigmoid, ReLU and GELU did not help either.

4.7 Are the results robust?

As far as we checked, yes. None of these checks decides a hypothesis, and the settings checks use one seed.

- *By task:* within classification or regression alone, no comparison is significant after correction. Neither the extra metrics the instructor asked for (accuracy, ROC AUC, mAP, R^2) nor the plan's secondary metrics show a gain (Table B1).
- *At a fixed depth:* with every cascade cut at layer 1, 2 or 3, no variant beats the base cascade. Choosing products by the layer error still beats choosing by the target there.
- *Depth rule:* the cascade often predicts from layer 1 (43% of runs), before the XP, uncertainty and monotone columns come in. The fixed-depth check shows this does not hide a gain. The rule itself stops too early: a fixed depth of 2 would have been better on 19 of 32 datasets.

- *Other settings:* we halved and doubled the number of products, used 4 + 4 forests, 500 trees, no binning and 5 inner folds. In median, residual-XX stays within about 1% of the base cascade, and the uncertainty and monotone columns within 0.1%.

4.8 How do the other models compare?

Tuned LightGBM ranks best of all 11 methods (Figure 3) and beats the base cascade on 21 of 32 datasets, though not significantly. The base cascade in turn beats a single random forest with as many trees as one layer. Every cascade variant stays inside the band of no clear difference from the base cascade. Products do not help a single forest either: RF with products is worse than plain RF on 24 of 32 datasets. Means and standard deviations per dataset are in Table B2.

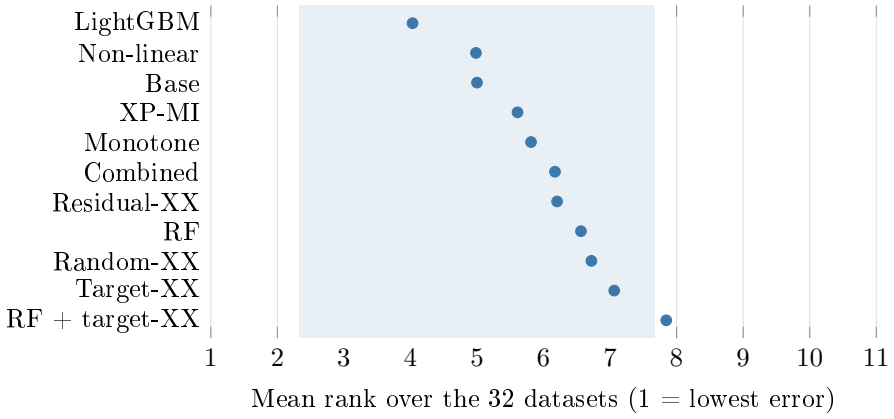


Figure 3: Mean rank of each method. A method inside the shaded band cannot be told apart from the base cascade (Nemenyi test [11], descriptive). Only RF + target-XX falls outside it; LightGBM's lead is within the band too.

5 Does the idea work at all?

Yes, on data built for it. A negative result says little if the method could never work, so after the main run we added a *positive control*: two synthetic datasets with a planted interaction of exactly the kind the products compute. Both have 300 rows and six random features. In one, the class is the sign of $x_1 x_2$, with 10% of labels flipped, so 90% is the best possible score. In the other, the target is $x_1 + 3x_2 x_3$ plus noise. Here the products cut the base cascade's error by about 40% (Figure 4), and both residual and target selection found the right pair at the first layer every time. In a follow-up with 1,000 rows we hid the same kind of target among 30 features, nearly all of them noise. Residual selection still found the pair every time and cut the error by more than half, more than LightGBM managed.

6 Why the products do not help on real data

After seeing the results we ran follow-up experiments to find out why. They were chosen after the fact, and most use one seed, so they are hints, not proof. Before each new run we wrote down what result would count as a yes.

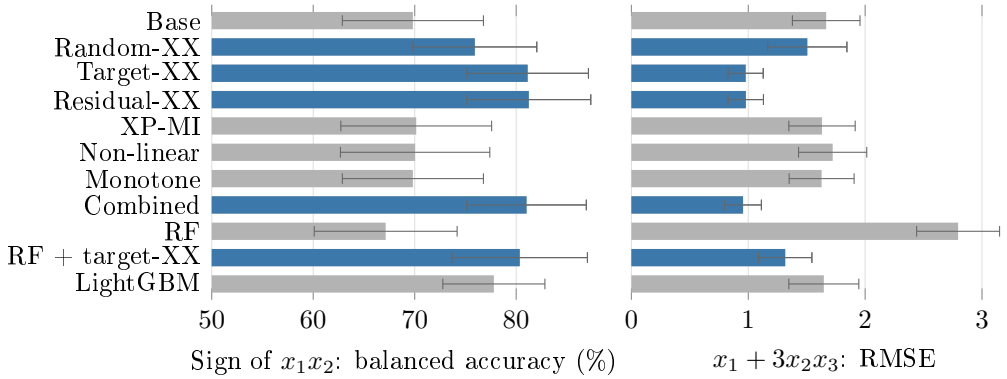


Figure 4: Positive control: mean over 15 runs, whiskers show the standard deviation. Blue: methods that add feature products. Higher balanced accuracy and lower RMSE are better; balanced accuracy starts at chance level (50%) and can reach at most about 90% here.

Extra columns cost more than the products bring. We took the chosen products and shuffled their rows: the same values, but no information about the row. These shuffled products made the cascade worse on 26 of 32 datasets, by a median 3%. That is the cost of extra columns that carry nothing. The real products beat the shuffled ones on 25 datasets, so they are better than noise. But randomly chosen products beat the shuffled ones just as often (26 datasets). The gap therefore does not show that our chosen products add something the cascade lacks; it shows that a real function of the features hurts less than noise. What is left is a small net loss (Figure 5). Two simpler explanations did not hold up: the cost stays when the original columns keep their old chance of being tried at a split, and products of uncentred features do no better.

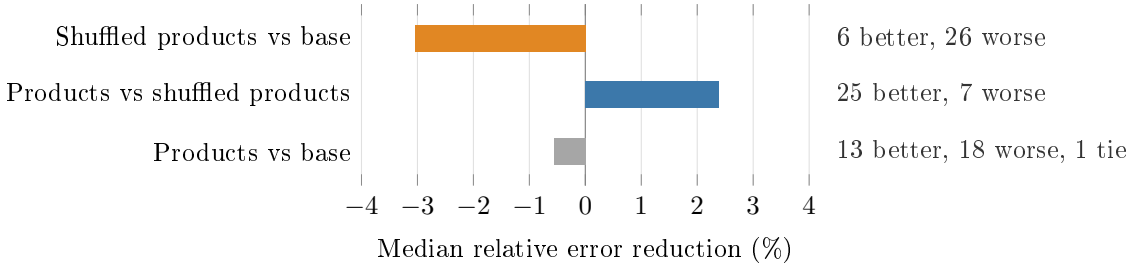


Figure 5: The chosen products, the same products shuffled, and the base cascade (one seed, 32 datasets). Shuffled products are the chosen products with their rows permuted. Right: datasets where the first method's error is lower or higher. These are three separate comparisons, so the medians do not add up.

A weak interaction is enough. We took 13 of the regression datasets, added two random columns u_1, u_2, and added a multiple of u_1u_2 to the target, so that this product explains 0, 6, 20 or 50% of the target's variance. This is the easiest case for the method: an exact product of two independent columns. Our bar, set before the run: products help if residual-XX beats the base cascade in median and on at least 9 of the 13 datasets. They did from 6% on, only slightly at first, and the gain grew with the strength of the product (Figure 6). We had expected them to help only from 20%.

A measure of interaction does not predict where products help. As a rough measure of interaction in each real dataset we took how much LightGBM gains from deeper trees over stumps. This noisy measure does not predict where products help. One

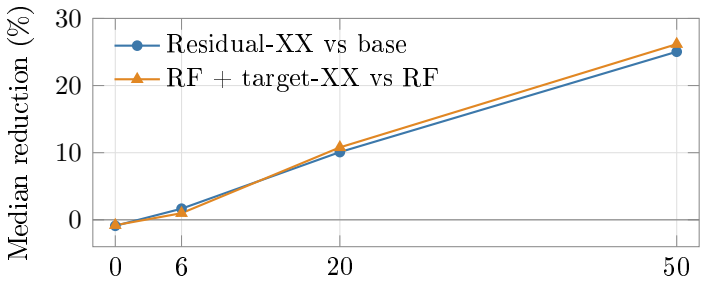


Figure 6: A planted product of growing strength on 13 regression datasets (one seed). The two lines nearly coincide: a single forest gains as much from the products as the cascade. Wins out of 13 at 0, 6, 20 and 50%: residual-XX against the base cascade 3, 11, 13, 13; RF + target-XX against RF 3, 8, 12, 13.

possible reading: the real datasets do have interactions, but not simple products of two features.

Depth already does part of the job. In the main runs, the products cut the first layer's error by more than 2% on 7 datasets. On 6 of them the base cascade's own later layers cut it by more, without any products.

7 Conclusions

Adding columns between the layers of Deep Forest did not make it better on our 32 datasets. None of the five hypotheses is supported, and for each kind of column a typical gain above about 0.7% is unlikely on data like ours. The idea is not broken. On data built for it, with an exact product of two features in the target, the products find that product: they cut the error sharply when it is strong and still help a little when it is weak. Follow-up runs suggest why this does not carry over to real data: the extra columns cost more than they bring, and the cascade's own depth already does part of what the products would do. Choosing products again at every layer lost less than keeping target-chosen products fixed, but did no better than random products, and adding none was better still.

Limitations.

- The follow-up runs (Section 6) were chosen after seeing the results. They and the settings checks use one seed, and results outside the six planned tests are mostly not corrected for multiple tests.
- The relative reduction exaggerates changes when errors are close to zero (Tic-Tac-Toe in Figure 2). The Wilcoxon test uses ranks, which limits the weight of such a dataset.
- Against the plan, three datasets kept categories stored as numbers as numeric columns. One-hot encoding them changes no conclusion (one seed).
- We add columns only between layers. The instructor suggested interactions within a layer, which we have not tried.

Further work.

- Interactions within a layer (what exactly this means is still to be agreed).
- Adding products only when they beat shuffled ones on held-out predictions.
- A better depth rule: the current one stops too early, and 5 inner folds instead of 3 made every cascade we tried more accurate.

8 Reproduction

Code, every setting with its source, run summaries and generated reports are at github.com/wojtke/deep-forests. With `uv` installed, `uv run dfx data` downloads the datasets and checks their hashes, `uv run dfx run main --workers 8` runs the main experiment (it can be stopped and resumed), and `uv run dfx report main` writes its tables and figures; `sensitivity` and `positive` work the same way. The follow-up runs have their own experiments and scripts, listed in the README. The outer splits use seeds 0, 1 and 2, and every other seed is derived from the task. The planned runs were made with the code of the repository's first commit, and the README explains how to resume or check them from there. The per-run files with every layer's predictions are not yet published. The long report [2] is in the repository too.

9 AI assistance

Claude Code (Opus 5.5) wrote the experiment code to the plan's specification, ran the experiments and analyses, proposed the follow-up runs and drafted this report. Separate Claude and Codex reviewers checked the code, the results and the text. The authors decided what to run and checked the results and the text.

References

[1] W. Jasiński and S. Adamiec. Non-linear and Interaction Features for Deep Forest: final plan. Ensemble Learning – Research Project, Warsaw University of Technology, 20 October 2026.

[2] W. Jasiński and S. Adamiec. Non-linear and Interaction Features for Deep Forest: results report. 9 November 2026.

[3] Z.-H. Zhou and J. Feng. Deep Forest. *National Science Review*, 6(1):74–86, 2019.

[4] LAMDA-NJU. Deep Forest (DF21). Python package `deep-forest`, version 0.1.7.

[5] R. Blum, M. Hiabu, E. Mammen, and J. T. Meyer. Pure interaction effects unseen by Random Forests. *Comput. Stat. Data Anal.*, 212:108237, 2025.

[6] Y.-H. Chen, S.-H. Lyu, and Y. Jiang. Improving Deep Forest by Exploiting High-order Interactions. *IEEE ICDM*, 1030–1035, 2021.

[7] Z. Lin, Y. Xu, K. Liu, and L. Chen. MDGP-forest: A novel deep forest for multi-class imbalanced learning based on multi-class disassembly and feature construction enhanced by genetic programming. *Pattern Recognition*, 170:112070, 2026.

[8] Q. Tan, Y. Wen, Y. Xu, K. Liu, S. He, and X. Bo. Multi-view uncertainty deep forest: An innovative deep forest equipped with uncertainty estimation for drug-induced liver injury prediction. *Information Sciences*, 667:120342, 2024.

[9] G. Yu, J. Bien, and R. Tibshirani. Reluctant Interaction Modeling. arXiv:1907.08414v3, 2023.

[10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*, 2017.

[11] J. Demšar. Statistical Comparisons of Classifiers over Multiple Data Sets. *JMLR*, 7:1–30, 2006.

[12] M. Hollander, D. A. Wolfe, and E. Chicken. *Nonparametric Statistical Methods*, 3rd edition, chapter 3. Wiley, 2014.

A Settings and datasets

Table A1: Settings. m: columns a layer receives; d: input features after encoding. Same for every cascade variant; not tuned.

Setting	Value	Source
Forests per layer	2 random forests + 2 extremely randomized forests	DF21 [4]
Trees per forest	100, fully grown	DF21
Features tried per split	$\sqrt{m}$	DF21
Column binning	255 quantile bins	DF21
Layer predictions for training rows	3 inner folds (DF21: out-of-bag)	Zhou & Feng [3]
Depth	add layers until 2 in a row bring no gain, at most 20; predict with the best layer	DF21
Depth score	balanced accuracy (DF21: accuracy); squared error in regression	ours
Products added, K	$\min(d, \lfloor d(d-1)/8 \rfloor)$	ours
Single forest (RF)	400 trees, as many as one layer	ours
LightGBM	10 random settings (learning rate, depth, subsampling, L2 penalty) on 3 inner folds; up to 1,000 trees with early stopping	ours
Rows per dataset	at most 20,000 (larger sets subsampled once)	ours
Sensitivity checks	$\lceil K/2 \rceil$ and $2K$ products; 4 + 4 forests; 500 trees; no binning (sigmoid, ReLU, GELU); 5 inner folds; one seed each	plan [1]

Table A2: The 32 datasets: rows × input features before our exclusions and encoding. Names link to the source.

Classification	Rows × feat.	Regression	Target	Rows × feat.
Sonar	208 × 60	Abalone	rings	4,177 × 8
Spambase	4,601 × 57	Auto MPG	mpg	398 × 7
QSAR Biodegradation	1,055 × 41	Automobile	price	205 × 25
Steel Plates Faults	1,941 × 27	Communities & Crime	ViolentCrimesPerPop	1,994 × 127
Statlog Vehicle	846 × 18	Servo	`class` (continuous)	167 × 4
Statlog Heart	270 × 13	Energy Efficiency	Y1 (heating load)	768 × 8
Tic-Tac-Toe Endgame	958 × 9	Wine Quality (red)	quality	1,599 × 11
Page Blocks	5,473 × 10	Concrete Slump	compressive strength	103 × 7
German Credit	1,000 × 20	Real Estate	price per unit area	414 × 6
Cardiotocography (NSP)	2,126 × 21	Parkinsons Telemonitoring	total UPDRS	5,875 × 19
Climate Crashes	540 × 18	Student Performance (maths)	G3	395 × 30
Phishing Websites	11,055 × 30	Superconductivity	critical temperature	21,263 × 81
SPECTF Heart	267 × 44	Bike Sharing (hourly)	count	17,379 × 13
Default of Credit Card Clients	30,000 × 23	Combined Cycle Power Plant	energy output	9,568 × 4
Bank Marketing	45,211 × 16	Air Quality	CO(GT)	9,357 × 14
Adult	48,842 × 14	California Housing	median house value	20,640 × 8

B Results by task and per dataset

Table B1: The six comparisons by task and metric: wins/losses of the first method over the 16 datasets of each task; the rest are ties. $^{*}p < 0.05$, $^{**}p < 0.01$, Wilcoxon test without correction. For balanced accuracy and RMSE, Holm's correction over the 12 comparisons gives $p \geq 0.39$.

	Classification						Regression		
	Bal. acc.	Accuracy	ROC AUC	mAP	Log-loss	Macro-F1	RMSE	MAE	R^2
H1	7/9	8/8	4/12*	6/10	5/11	9/7	7/9	6/10	6/10
H2a	8/8	7/8	5/11	5/11	6/10	8/8	8/8	7/9	7/9
H2b	11/5	11/5	8/8	8/8	11/5	10/6	9/4	8/5	8/5
H3	7/8	9/6	9/7	8/8	3/13*	9/7	2/9*	3/8*	2/9*
H4	10/6	10/6	9/7	10/6	9/7	10/6	4/6	5/5	5/5
H5	7/9	7/9	2/14**	2/14*	1/15**	8/8	1/9	1/9	1/9

Table B2: Primary metric per dataset: mean ± standard deviation over 15 runs; best mean in bold. Top: balanced accuracy (%), higher is better. Bottom: RMSE in the target's units, lower is better.

Dataset	Base	Random-XX	Target-XX	Residual-XX	XP-MI	Non-linear	Monotone	Combined	RF	RF + target-XX	LightGBM
Sonar	84.5 ± 5.0	85.5 ± 6.0	82.4 ± 4.4	82.8 ± 4.0	84.2 ± 5.1	84.8 ± 5.1	83.8 ± 4.5	83.3 ± 5.0	83.9 ± 5.2	81.7 ± 5.9	**85.7 ± 5.0**
Spambase	94.7 ± 0.8	94.7 ± 0.9	94.9 ± 0.8	94.9 ± 0.7	94.5 ± 0.9	94.7 ± 0.8	94.6 ± 0.7	**94.9 ± 0.8**	94.6 ± 0.9	94.3 ± 1.0	94.8 ± 1.1
QSAR Biodeg.	84.0 ± 3.5	84.2 ± 3.5	84.1 ± 3.6	84.0 ± 3.5	84.4 ± 3.8	84.4 ± 3.6	84.1 ± 3.5	84.1 ± 3.4	84.0 ± 3.6	83.7 ± 3.1	**84.5 ± 3.8**
Steel Plates	78.2 ± 3.9	77.6 ± 3.5	76.6 ± 2.7	77.3 ± 3.2	78.1 ± 3.5	78.4 ± 3.7	78.2 ± 3.9	77.6 ± 3.3	78.6 ± 3.1	76.6 ± 2.9	**80.2 ± 2.0**
Statlog Vehicle	76.9 ± 3.5	76.1 ± 2.6	75.9 ± 1.9	75.4 ± 2.6	77.0 ± 3.6	**77.4 ± 3.0**	76.2 ± 3.3	75.5 ± 2.1	74.6 ± 2.5	75.4 ± 2.8	77.2 ± 3.1
Statlog Heart	82.4 ± 4.6	81.9 ± 4.9	82.2 ± 4.8	82.7 ± 4.6	**83.4 ± 3.7**	82.9 ± 4.6	83.0 ± 4.1	82.6 ± 4.5	81.3 ± 4.3	80.9 ± 4.7	82.3 ± 3.1
Tic-Tac-Toe	98.6 ± 1.1	98.0 ± 1.2	97.8 ± 1.8	97.7 ± 1.9	98.9 ± 1.0	98.5 ± 1.3	98.5 ± 1.1	97.7 ± 1.9	98.3 ± 0.9	96.3 ± 1.7	**99.5 ± 0.8**
Page Blocks	85.8 ± 4.9	85.0 ± 7.3	85.1 ± 6.5	85.2 ± 6.1	86.0 ± 5.6	86.0 ± 5.3	**86.2 ± 5.4**	85.0 ± 6.3	84.3 ± 5.7	85.5 ± 5.0	84.6 ± 4.9
German Credit	66.2 ± 3.7	65.7 ± 4.0	66.6 ± 2.9	66.5 ± 3.0	66.2 ± 3.7	66.4 ± 3.8	66.4 ± 3.8	66.5 ± 2.9	65.2 ± 3.2	**67.0 ± 3.2**	66.4 ± 2.8
Cardiotocography	84.3 ± 3.2	84.3 ± 2.7	82.8 ± 3.4	83.1 ± 3.4	83.4 ± 2.5	**84.5 ± 2.9**	84.1 ± 2.7	83.1 ± 3.6	82.4 ± 3.1	80.9 ± 3.9	83.8 ± 2.6
Climate Crashes	76.2 ± 9.3	73.0 ± 9.0	76.2 ± 7.7	78.2 ± 7.5	75.9 ± 8.6	75.5 ± 9.2	75.1 ± 8.6	**78.5 ± 7.5**	56.6 ± 4.8	62.9 ± 7.8	73.1 ± 8.4
Phishing	95.3 ± 0.6	95.0 ± 0.7	95.2 ± 0.5	95.2 ± 0.5	95.3 ± 0.6	95.3 ± 0.6	95.3 ± 0.6	95.2 ± 0.5	**95.4 ± 0.6**	95.3 ± 0.6	95.4 ± 0.7
SPECTF Heart	64.1 ± 7.1	62.6 ± 5.7	59.5 ± 6.5	61.9 ± 7.3	65.0 ± 6.3	63.6 ± 6.7	**65.5 ± 8.0**	62.2 ± 7.9	60.1 ± 5.0	57.4 ± 5.2	63.4 ± 5.5
Credit Default	66.0 ± 0.8	**66.1 ± 1.1**	66.0 ± 1.0	66.1 ± 1.0	66.0 ± 1.0	66.0 ± 0.8	65.9 ± 0.8	66.0 ± 1.0	65.9 ± 0.7	65.9 ± 0.8	65.9 ± 0.9
Bank Marketing	60.3 ± 0.8	60.3 ± 0.8	60.6 ± 0.9	60.8 ± 1.0	60.2 ± 0.7	60.1 ± 0.8	60.1 ± 0.7	60.6 ± 1.0	60.7 ± 0.7	**61.1 ± 0.8**	60.8 ± 0.7
Adult	77.0 ± 0.9	76.7 ± 0.7	77.0 ± 0.8	77.1 ± 0.7	76.5 ± 1.0	76.6 ± 0.7	76.9 ± 0.8	76.9 ± 0.8	76.9 ± 0.7	76.6 ± 0.6	**78.9 ± 0.7**
Abalone	2.15 ± 0.05	2.16 ± 0.05	2.17 ± 0.05	2.17 ± 0.05	2.15 ± 0.05	2.15 ± 0.05	2.15 ± 0.05	2.17 ± 0.05	2.16 ± 0.04	2.18 ± 0.05	**2.13 ± 0.05**
Auto MPG	2.81 ± 0.34	2.78 ± 0.35	2.93 ± 0.39	2.77 ± 0.44	2.82 ± 0.34	2.81 ± 0.34	2.82 ± 0.33	2.79 ± 0.39	**2.73 ± 0.34**	2.88 ± 0.41	2.80 ± 0.30
Automobile	2,191 ± 554	2,334 ± 653	2,407 ± 667	2,406 ± 672	2,297 ± 678	2,245 ± 556	2,087 ± 472	2,436 ± 683	**2,066 ± 529**	2,183 ± 489	2,475 ± 521
Communities	0.136 ± 0.005	0.137 ± 0.005	0.137 ± 0.005	0.137 ± 0.005	0.136 ± 0.005	0.136 ± 0.005	0.136 ± 0.005	0.137 ± 0.005	0.135 ± 0.005	0.137 ± 0.005	**0.135 ± 0.005**
Servo	0.579 ± 0.211	0.597 ± 0.206	0.566 ± 0.194	0.567 ± 0.197	0.579 ± 0.212	0.609 ± 0.230	0.588 ± 0.247	0.607 ± 0.218	0.620 ± 0.161	**0.542 ± 0.152**	0.612 ± 0.173
Energy Efficiency	0.541 ± 0.151	0.545 ± 0.157	0.538 ± 0.171	0.535 ± 0.165	0.546 ± 0.151	0.544 ± 0.142	0.546 ± 0.149	0.545 ± 0.159	0.651 ± 0.161	0.529 ± 0.171	**0.406 ± 0.074**
Wine Quality	0.629 ± 0.033	0.634 ± 0.036	0.636 ± 0.037	0.635 ± 0.036	0.629 ± 0.033	0.629 ± 0.033	0.629 ± 0.033	0.635 ± 0.036	**0.629 ± 0.032**	0.634 ± 0.036	0.631 ± 0.031
Concrete Slump	3.71 ± 0.66	3.87 ± 0.62	4.00 ± 0.60	3.98 ± 0.66	3.79 ± 0.64	3.81 ± 0.65	3.72 ± 0.64	4.01 ± 0.64	3.85 ± 0.69	3.91 ± 0.59	**3.02 ± 0.41**
Real Estate	7.66 ± 1.66	**7.62 ± 1.72**	7.68 ± 1.67	7.74 ± 1.67	7.67 ± 1.73	7.62 ± 1.67	7.66 ± 1.66	7.64 ± 1.68	7.90 ± 1.65	7.98 ± 1.70	7.72 ± 1.75
Parkinsons	11.3 ± 1.6	11.3 ± 1.5	11.3 ± 1.6	11.3 ± 1.6	11.5 ± 1.7	11.4 ± 1.7	11.4 ± 1.6	11.3 ± 1.7	11.9 ± 1.3	11.9 ± 1.4	**10.9 ± 1.8**
Student Perf.	4.10 ± 0.22	4.09 ± 0.21	4.10 ± 0.23	4.10 ± 0.23	4.08 ± 0.21	4.10 ± 0.22	4.10 ± 0.22	4.09 ± 0.22	**3.98 ± 0.22**	4.08 ± 0.24	4.04 ± 0.25
Superconductivity	9.85 ± 0.37	9.82 ± 0.41	9.81 ± 0.43	9.76 ± 0.44	9.87 ± 0.39	9.78 ± 0.40	9.87 ± 0.39	9.73 ± 0.46	**9.57 ± 0.44**	9.61 ± 0.47	9.71 ± 0.45
Bike Sharing	49.6 ± 2.0	50.5 ± 2.2	58.4 ± 2.2	50.8 ± 2.0	50.8 ± 1.9	49.5 ± 2.0	50.1 ± 2.0	49.7 ± 2.2	63.0 ± 2.0	72.1 ± 2.3	**46.4 ± 1.6**
Power Plant	3.31 ± 0.08	3.37 ± 0.08	3.41 ± 0.07	3.41 ± 0.07	3.31 ± 0.08	3.31 ± 0.08	3.31 ± 0.08	3.41 ± 0.07	3.23 ± 0.08	3.33 ± 0.08	**3.15 ± 0.11**
Air Quality	0.453 ± 0.036	0.456 ± 0.035	0.452 ± 0.036	0.452 ± 0.036	0.453 ± 0.036	0.453 ± 0.036	0.453 ± 0.036	0.452 ± 0.036	**0.450 ± 0.038**	0.452 ± 0.038	0.461 ± 0.045
California Housing	0.478 ± 0.006	0.492 ± 0.008	0.479 ± 0.006	0.481 ± 0.006	0.476 ± 0.006	0.477 ± 0.006	0.481 ± 0.006	0.480 ± 0.006	0.490 ± 0.006	0.490 ± 0.005	**0.445 ± 0.008**